

A Mechanized Account of the QUPIT Agda Development

Deriving Box Relations for Odd-Prime-Dimensional Clifford Circuits

Anonymous Author(s)

Abstract

This paper explains the QUPIT Agda repository that accompanies the paper “A Complete and Natural Rule Set for Multi-Qudit Clifford Circuits in All Odd Prime Dimensions.” The development formalizes Clifford circuit syntax as words over generators, equips these words with presentation-generated congruence relations, defines modular arithmetic over finite fields $\mathbb{Z}_p$, and uses those ingredients to derive box-style rewrite relations for one-, two-, and three-qupit fragments. The repository is not merely a transcription of circuit diagrams: its core abstractions separate raw word syntax, presentation closure, algebraic normal forms, Pauli action, and checked equational derivations. This separation is what makes the proof scripts usable: large circuit equalities are decomposed into finite-field arithmetic, associativity normalization, lifting lemmas for extra wires, and small reusable Clifford identities.

The paper is written as an artifact-oriented account of the code. It identifies the trusted and untrusted boundaries in the development, summarizes the implemented relation sets, explains how boxes are encoded and interpreted back into circuits, and maps the public box-relation statements in `BoxRelations.agda` to their proof modules under `N/BR`. Several older or exploratory modules in the repository contain unfinished holes; following the artifact itself, we treat those modules as outside the described proof spine.

CCS Concepts: • **Theory of computation** → **Program verification; Equational logic and rewriting;** • **Hardware** → *Quantum computation*.

Keywords: Agda, Clifford circuits, qudits, presentations, mechanized mathematics, rewriting

ACM Reference Format:

Anonymous Author(s). 2027. A Mechanized Account of the QUPIT Agda Development: Deriving Box Relations for Odd-Prime-Dimensional Clifford Circuits. In *Proceedings of ACM SIGPLAN Symposium on Principles of Programming Languages (POPL 2027)*. ACM, New York, NY, USA, 8 pages.

1 Introduction

Reasoning about Clifford circuits is algebraic, but the algebra is large enough that informal derivations become brittle. A typical box relation states that a primitive gate can be pushed

through a parameterized circuit pattern, producing another parameterized pattern and a residual gate sequence. Even when the high-level rule is conceptually simple, a proof may require case analysis over finite field elements, repeated use of modular arithmetic, and long associativity rearrangements of circuit words. The QUPIT repository uses Agda to make these derivations checkable.

The development targets Clifford circuits over odd prime dimensions. The prime is represented in the code by parameters `p-2 : Nat` and `p-prime : Prime (2+ p-2)`; the actual dimension is $p = 2 + p-2$. The finite field carrier $\mathbb{Z}_p$ is implemented with `Fin p`, and all circuit parameters used by the box definitions are elements of this finite type. This choice lets the development use ordinary dependent types for field-indexed circuits while keeping the prime assumption explicit in every module that depends on it.

At the top level, the README says that the repository accompanies a paper on a complete and natural rule set for multi-qudit Clifford circuits. The existing documentation, `doc/Readme.pdf`, gives a compact explanation of the central proof idea: instead of doing the box-relation calculations by hand, define the relevant presentations inside Agda and let the type checker validate the derivations. This paper expands that documentation into a POPL-style artifact paper, with an emphasis on the engineering and proof architecture visible in the code.

The most important constraint on this account is fidelity to the repository. The code base contains more than one layer of work: generic presentation infrastructure, active Clifford and box-relation modules, simplified and derived relation sets, and older experiments. Some experimental modules contain unfinished holes. We therefore describe the implemented proof spine and explicitly separate it from unused or unfinished material.

Contributions. This paper makes four contributions.

- It gives a module-by-module account of the QUPIT mechanization, from words and congruence closures to Clifford generators and box relations.
- It explains why the repository represents circuits as binary words rather than lists, and how associativity is recovered by a presentation-level congruence and list-based normalization lemmas.
- It documents the relation between the comprehensive `N/Symplectic.agda` relation set, the simplified

N/Symplectic-Simplified.agda relation set, and the derived relation infrastructure used by Pauli actions.

- It maps the public theorem index in BoxRelations.agda to the one-, two-, and three-qubit proof modules under N/BR.

2 Mathematical Background

This section records the mathematics that the development mechanizes. None of it is new; it fixes notation and explains why a presentation of words, quotiented by a congruence, is an adequate model of Clifford equality in odd prime dimension. Throughout, p is an odd prime and $\mathbb{Z}_p = \mathbb{Z}/p\mathbb{Z}$ is the field with p elements.

2.1 The qudit Pauli group

Fix a primitive p -th root of unity $\omega = e^{2\pi i/p}$ and let the shift and clock operators act on $\mathbb{C}^p$ by

$$X|j\rangle = |j+1 \bmod p\rangle, \quad Z|j\rangle = \omega^j|j\rangle.$$

They satisfy $X^p = Z^p = I$ and the Weyl commutation relation $ZX = \omega XZ$. For $(a, b) \in \mathbb{Z}_p^2$ the Weyl operator $W(a, b) = \omega^{-2^{-1}ab} X^a Z^b$ is well defined because 2 is invertible in $\mathbb{Z}_p$; this is one of the places where oddness of p is essential [10, 11]. The n -qudit Pauli (or Heisenberg–Weyl) group $\mathcal{P}_n$ is generated by the operators $W(a, b)$ on each wire together with the central phases $\langle \omega \rangle$. Two Weyl operators commute up to a phase governed by the symplectic form: writing $v = (a, b)$ and $v' = (a', b')$,

$$W(v)W(v') = \omega^{\langle v, v' \rangle} W(v')W(v), \quad \langle v, v' \rangle = a'b - ab'.$$

This pairing is exactly sform1 of N/Pauli.agda up to the sign convention [5, 7].

2.2 The Clifford group and its symplectic quotient

The n -qudit Clifford group C_n is the normalizer of $\mathcal{P}_n$ in the unitary group $U(p^n)$, taken modulo global phase. Conjugation by a Clifford operator sends Weyl operators to Weyl operators, hence induces a $\mathbb{Z}_p$ -linear map on the label space $\mathbb{Z}_p^{2n}$. Because conjugation preserves Weyl commutators, that map preserves $\langle \cdot, \cdot \rangle$: it is symplectic. This is the content of the short exact sequence

$$1 \longrightarrow \mathcal{P}_n / \langle \omega \rangle \longrightarrow C_n / \langle \omega \rangle \longrightarrow \mathrm{Sp}(2n, \mathbb{Z}_p) \longrightarrow 1.$$

For odd p the sequence splits: every symplectic matrix is realized by a Clifford unitary, the splitting being the Weil (metaplectic) representation over a finite field [10–12]. Consequently, up to Paulis and phases, the Clifford group is $\mathrm{Sp}(2n, \mathbb{Z}_p)$. This is the structural fact that justifies the design of the repository: the symbolic congruence on circuit words is sound for Clifford equality precisely because both sides descend to the same symplectic matrix, and completeness amounts to the surjectivity above [7–9].

2.3 The symplectic group over $\mathbb{Z}_p$

Let Ω be the standard symplectic form on $\mathbb{Z}_p^{2n}$. The symplectic group is

$$\mathrm{Sp}(2n, \mathbb{Z}_p) = \{ M \in \mathrm{GL}_{2n}(\mathbb{Z}_p) \mid M^\top \Omega M = \Omega \},$$

and its order is

$$|\mathrm{Sp}(2n, \mathbb{Z}_p)| = p^{n^2} \prod_{i=1}^n (p^{2i} - 1).$$

For $n = 1$ the symplectic condition reduces to $\det M = 1$, so $\mathrm{Sp}(2, \mathbb{Z}_p) = \mathrm{SL}_2(\mathbb{Z}_p)$, of order $p(p^2 - 1)$ [30]. Finite presentations of these groups, of the kind the development reconstructs at the level of circuit words, are classical in combinatorial and computational group theory [25, 27, 28].

2.4 Generators as symplectic matrices

Write each transformation as a matrix acting on column vectors $(a, b)^\top$, matching the one-qubit actions of N/Action.agda. The phase gate, Hadamard, and multiplier act by

$$\rho(S^k) = \begin{pmatrix} 1 & 0 \\ k & 1 \end{pmatrix}, \quad \rho(H) = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}, \quad \rho(M(x)) = \begin{pmatrix} x^{-1} & 0 \\ 0 & x \end{pmatrix},$$

each of determinant 1, hence elements of $\mathrm{SL}_2(\mathbb{Z}_p)$. The one-qubit relations of N/Symplectic.agda are then literally matrix identities over $\mathbb{Z}_p$:

$$\rho(S)^p = I, \quad \rho(H)^4 = I, \quad \rho(H)^2 = \rho(M(-1)) = -I, \quad \rho(M(x))\rho(M(y)) = \rho(M(xy)), \quad \rho(M(x))\rho(M(y)) = \rho(M(y))\rho(M(x))$$

For instance $\rho(H)^2 = -I = \mathrm{diag}(-1, -1) = \rho(M(-1))$ recovers the simplified axiom $H^2 = M(-1)$, and the conjugation identity recovers $M(x)S = S^{x^2}M(x)$. On two wires $\rho(CZ)$ is the symplectic shear

$$\rho(CZ) : (a, b), (a', b') \longmapsto (a, b + a'), (a', b' + a),$$

which preserves the form and commutes with sufficiently shifted single-wire gates—exactly the structural relations the code records as the Selinger relations [16].

2.5 Boxes as symplectic coordinates

The box families A, B, D, E and the normal forms NF(n) of N/Boxes.agda form a coordinate system for elements of the symplectic group: each box stores the field parameters of one elementary symplectic factor, and a normal form NF(n) is an ordered product of such factors, one layer per wire block. A box relation $[x] \cdot g \approx \mathrm{dir} \cdot [x']$ then says that multiplying a normal-form factor by a generator g can be re-expressed as a direction word dir times an updated factor $[x']$; read through ρ , it asserts the equality of the two symplectic matrices. Pushing a generator past the factors of a normal form and recomputing the parameters by finite-field arithmetic is therefore a constructive form of the Bruhat-style factorization of $\mathrm{Sp}(2n, \mathbb{Z}_p)$ [28, 29]. This is the geometric meaning of the otherwise syntactic derivations in N/BR.

3 Repository Overview

Table 1 summarizes the active parts of the project that this paper discusses. The table follows the structure of the repository rather than imposing a separate mathematical organization.

The generic presentation layer is intentionally independent of Clifford circuits. It can be read as a small library for monoid presentations: a presentation is a generator type X , a relation $\Gamma : \text{Word} X \rightarrow \text{Word} X \rightarrow \text{Set}$, and the congruence closure of Γ . The Clifford-specific layer instantiates this infrastructure with generators H , S , CZ , and lifting constructors that move generators to later wires.

4 Words Rather Than Lists

The basic syntax of circuits is defined in `Word/Base.agda`. The definition is small but central:

```
data Word (X : Set) : Set where
  [_]^w : X -> Word X
  epsilon : Word X
  _bullet_ : Word X -> Word X -> Word X
```

In the actual code the constructor names use Unicode: $[_]^w$, `epsilon` is written ϵ , and `_bullet_` is written $\bullet$. The paper uses ASCII names in listings where that improves portability.

The key design choice is that `Word X` is a binary tree. A list of gates would quotient away the parse tree of composition too early: the expression $u \cdot v$ carries the fact that it was built from left operand u and right operand v , while the flattened list of gates does not. Many equational proofs need to rewrite only the left or right side of a product; keeping the tree makes such proof steps structurally visible to Agda.

Associativity is not built into the syntax. Instead, it is added as a proof rule in the congruence closure. This is a familiar tradeoff in mechanized algebra: raw syntax stays simple and inductive, while the semantic equality relation supplies associativity, units, and domain axioms.

The word module also defines powers:

$$w^0 = \epsilon, \quad w^1 = w, \quad w^{n+2} = w \cdot w^{n+1}.$$

Powers are used throughout the Clifford development for relations such as $S^p = \epsilon$, $CZ^p = \epsilon$, and $H^4 = \epsilon$, and also for parameterized circuits such as S^k and CZ^k .

5 Presentation Closure

The congruence closure is implemented in `Presentation/Base.agda`. For a relation Γ on words, the module defines an equality $\approx$ generated by reflexivity, symmetry, transitivity, congruence, monoid laws, and the axioms in Γ :

```
data _approx_ : WRel X where
  refl      : w approx w
  sym       : w approx v -> v approx w
  trans     : w approx v -> v approx u -> w approx u
  cong      : w approx w' -> v approx v' ->
```

```
      w bullet v approx w' bullet v'
  assoc     : (w bullet v) bullet u approx w bullet
              (v bullet u)
  left-unit : epsilon bullet w approx w
  right-unit : w bullet epsilon approx w
  axiom     : Gamma w v -> w approx v
```

The closure is the object that most later proofs inhabit. A theorem such as “pushing S through an E box” is an Agda term whose type is an equality in this closure. Setoid reasoning then provides the readable calculation style.

The module `Presentation/Properties.agda` packages the closure as an Agda setoid and proves that word composition forms a magma, a semigroup, and a monoid modulo $\approx$. It also supplies a practical associativity layer. The functions `to-list` and `from-list` flatten and rebuild words, and the lemma `by-assoc` proves that equal flattened lists imply equality of the original words modulo associativity. Later proof scripts use this infrastructure, directly or through tactics in `Presentation/Tactics.agda`, to avoid drowning in parenthesis management.

6 Finite Fields

The finite-field layer lives under $\mathbb{Z}_p$. The carrier of $\mathbb{Z}/n\mathbb{Z}$ is `Fin n`:

```
Z : Nat -> Set
Z n = Fin n
```

Nonzero elements are represented as dependent pairs:

```
Z* : Nat -> Set
Z* zero = Z zero
Z* (suc n) = Sigma[ a in Z (suc n) ] (a /= zero)
```

Addition and multiplication are implemented by converting to natural numbers, applying ordinary addition or multiplication, and reducing modulo the modulus. The file then proves the expected algebraic laws: associativity and commutativity of addition and multiplication, identities, additive inverse, distributivity, and additional lemmas about powers and inverses. For the Clifford development, the prime modulus modules expose the dimension

$$p = 2 + p-2$$

and field-specific operations such as inverse on nonzero elements.

This layer matters because the box relations are parameterized by field elements. For example, an A contains a nonzero pair (a, b) , an E contains one field element, and the interpretation of boxes uses expressions such as $-b/a$. In the code, $-b/a$ is not a partial operation; it is introduced only in branches where the denominator is accompanied by a proof that it is nonzero.

Table 1. Main modules used by the QUPIT development.

File or directory	Role
Word/Base.agda	Binary word syntax, powers, word maps, folds, and word relations.
Word/Properties.agda	Word-map laws and decidable equality for words.
Presentation/Base.agda	Congruence closure of a relation on words.
Presentation/Properties.agda	Setoid, magma, semigroup, monoid, and associativity tools.
Presentation/Morphism*.agda	Morphisms and identity/isomorphism infrastructure for presentations.
Presentation/CosetNF.agda	Generic coset and Reidemeister-Schreier-style normal-form machinery.
Zp/ModularArithmetic.agda	Modular arithmetic over $\text{Fin } n$; ring and field-like lemmas.
Zp/Fermats-little-theorem.agda	Prime-modulus and primitive-root support.
N/Symplectic.agda	Main generator set and comprehensive Clifford relations.
N/Symplectic-Simplified.agda	Paper-facing simplified relation set parameterized by a primitive root.
N/Symplectic-Derived.agda	Derived generators such as parameterized powers of H, S, CZ .
N/Iso-Sym-Derived.agda	Isomorphism between the basic and derived symplectic presentations.
N/Action.agda, N/Pauli.agda	Pauli vectors, symplectic form, and generator action.
N/Boxes.agda, N/LM-Sym.agda	Box parameter spaces and interpretation of boxes as circuit words.
N/BR/...	Concrete box-relation derivations and supporting calculations.
BoxRelations.agda	Public index of box-relation statements and proof pointers.

7 Clifford Generators

The main generator type is defined inside the module `N/Symplectic.agda`.

For n qubits, generators are indexed by the number of wires on which they are well-formed:

```

data Gen : Nat -> Set where
  H-gen  : {n : Nat} -> Gen (1+n)
  S-gen  : {n : Nat} -> Gen (1+n)
  CZ-gen : {n : Nat} -> Gen (2+n)
  _lift  : {n : Nat} -> Gen n -> Gen (suc n)

```

The real code writes `_lift` as a Unicode up-shift constructor. The lifting constructor shifts a generator onto later wires. Thus the first-wire gates are named directly, and gates on later wires are obtained by repeated lifting. Words are lifted with a map:

$$w^\uparrow = \text{wmap } (_ \uparrow) w.$$

The code also defines a down-shift operation, but in the active `N/Symplectic.agda` presentation `_down` is implemented as the identity on words. Several older comments show a more structural down-shift version, but the proof spine described here uses the current code.

The primitive circuits are:

$$S = [S\text{-gen}], \quad H = [H\text{-gen}], \quad CZ = [CZ\text{-gen}].$$

From these, the repository defines common derived circuits:

$$\begin{aligned}
S^{-1} &= S^{p-1}, \\
CZ^{-1} &= CZ^{p-1}, \\
CX &= H^{\downarrow 3} \cdot CZ \cdot H^{\downarrow}, \\
XC &= H^{\uparrow 3} \cdot CZ \cdot H^{\uparrow}.
\end{aligned}$$

The multiplicative gate $M(x)$, for $x \in \mathbb{Z}_p^*$, is encoded as

$$M(x) = S^x \cdot H \cdot S^{x^{-1}} \cdot H \cdot S^x \cdot H.$$

This definition appears both in the README example and in the main symplectic modules.

8 Comprehensive Relations

The relation family in `N/Symplectic.agda` is called `_QRel`, `_===`. It is indexed by the number of wires and is used as the axiom relation for the presentation closure. The implemented constructors include:

- one-qubit relations $S^p = \epsilon$, $H^4 = \epsilon$, $(SH)^3 = \epsilon$, and $HHS = SHH$;
- multiplicative-gate laws $M(x)M(y) = M(xy)$ and $M(x)S = S^{x^2}M(x)$;
- two-qubit scaling laws for $M(x)$ through CZ on either side;
- $CZ^p = \epsilon$ and commutation of CZ with lifted S -gates;
- Selinger-style two- and three-qubit relations named `selinger-c10` through `selinger-c15`;
- commutation rules saying sufficiently shifted generators commute with H, S , and CZ ;
- a congruence-lifting axiom `cong-up`, written `cong` followed by an up-arrow in the code, which lifts an equality on n wires to one on $n + 1$ wires.

The module also proves basic lifting lemmas. For example, if $w \approx v$ under the n -wire presentation, then $w^\uparrow \approx v^\uparrow$ under the $(n + 1)$ -wire presentation. In the code this is `lemma-cong-up`, written with an up-arrow in the actual identifier, proved by induction on the derivation of $w \approx v$.

9 Simplified and Derived Presentations

The repository distinguishes at least three related presentations. The comprehensive presentation in `N/Symplectic.agda` is convenient for proofs because it includes relations such as $M(x)M(y) = M(xy)$ for arbitrary nonzero x, y . The simplified presentation in `N/Symplectic-Simplified.agda` is closer to the paper-facing axiom set. It is parameterized by a primitive root $g \in \mathbb{Z}_p^*$ and a proof that every nonzero element is a power of g . Its one-qubit multiplicative axioms use $M(g)$ and powers of $M(g)$ rather than arbitrary $M(x)$.

The simplified relation family contains constructors such as:

$$\begin{aligned} S^p &= \epsilon, \\ H^2 &= M(-1), \\ M(g)^k &= M(g^k), \\ M(g)S &= S^{g^2}M(g). \end{aligned}$$

It also contains the same two- and three-qubit structural relations needed to reason about CZ, lifted gates, and shifted wires. This matches the README's single-quint example, where the primitive root is $2 \in \mathbb{Z}_5^*$, and the simplified axiom set derives the box relation for E .

The derived presentation in `N/Symplectic-Derived.agda` exposes parameterized generators, for example H^k , S^k , and CZ^k , as generators rather than only as powers of primitive words. The module `N/Iso-Sym-Derived.agda` defines maps between the basic and derived presentations:

$$f : \text{Gen}_{\text{basic}}(n) \rightarrow \text{Gen}_{\text{derived}}(n), \quad g : \text{Gen}_{\text{derived}}(n) \rightarrow \text{Word}(\text{Gen}_{\text{basic}}(n))$$

It then proves well-definedness lemmas for the maps and establishes isomorphism infrastructure. The module `N/Action-Sym.agda` uses the composition of these maps to define the Pauli action of the simplified presentation through the derived one.

10 Pauli Action

The semantic side of the development is represented by Pauli vectors. In `N/Pauli.agda`, a one-qubit Pauli is a pair of field elements:

$$\text{Pauli1} = \mathbb{Z}_p \times \mathbb{Z}_p.$$

An n -qubit Pauli is a vector of n such pairs. The symplectic form is:

$$\text{sform1}((a, b), (c, d)) = -ad + cb,$$

extended componentwise over vectors.

The module `N/Action.agda` defines the action of derived generators on Pauli vectors. The action of S^k maps (a, b) to $(a, b + ak)$; the action of H rotates (a, b) to $(-b, a)$ for exponent 1; and CZ^k updates the two adjacent components by adding cross terms:

$$(a, b), (a', b') \mapsto (a, b + a'k), (a', b' + ak).$$

Lifted generators act on the tail of the vector, leaving the first component unchanged.

This Pauli action is used by the normal-form modules. It lets the development state and prove facts of the form “there exists a word in normal form whose action sends one Pauli vector to another with the same symplectic invariant.” The one-qubit normal-form module `N/NF1.agda` is the clearest complete example: it contains theorems such as `Theorem-NF1`, `Theorem-MC`, and variants specialized to pZ -like inputs.

Symplectic invariance. The action just described is the matrix action ρ of Section 2: each generator acts by a matrix in $\text{Sp}(2n, \mathbb{Z}_p)$, and a word acts by the product of the matrices of its letters, so the whole congruence class of a word has a single symplectic image. Every generator preserves `sform1`, hence so does every derivable equality; the quantity $\langle \cdot, \cdot \rangle$ is the invariant carried along a derivation. The normal-form theorems are thus statements about orbits of $\text{Sp}(2n, \mathbb{Z}_p)$ acting on Pauli vectors, which is also the basis of the efficient classical description of stabilizer dynamics [6, 8].

11 Boxes and Normal Forms

The box syntax is defined in `N/Boxes.agda`. The definitions are small but heavily indexed:

$$\begin{aligned} A &= \{(a, b) \in \mathbb{Z}_p^2 \mid (a, b) \neq (0, 0)\}, \\ B &= \mathbb{Z}_p \times \mathbb{Z}_p, \\ D &= \mathbb{Z}_p \times \mathbb{Z}_p, \\ E &= \mathbb{Z}_p. \end{aligned}$$

The wider box families are indexed by arity. For example, `L(n)`, `M(n)`, `LM(n)`, and `NF(n)` are recursive families describing normal-form-shaped circuits over n wires. A normal form for $n + 1$ wires consists of a normal form on n wires and an LM layer on the last wire block:

$$\text{NF}(0) = \top, \quad \text{NF}(n + 1) = \text{NF}(n) \times \text{LM}(n + 1).$$

The box syntax itself is not a circuit. The interpretation appears in `N/LM-Sym.agda`, which maps boxes to words over the symplectic generator set. Selected definitions are:

$$\begin{aligned} E &= S^{-b}, \\ [(0, b)]^B &= Ex \cdot CX'^b, \\ [(0, b)]^D &= Ex \cdot CZ^{-b}. \end{aligned}$$

For nonzero first components, the interpretations introduce inverses in $\mathbb{Z}_p^*$. For instance, a nonzero $A(a, b)$ is interpreted using a^{-1} and $-b/a$. The code enforces the side condition by pattern matching on whether the first component is zero.

The vector box interpretations are also order-sensitive. The comment in `N/LM-Sym.agda` states that a vector of `B` boxes is in circuit order, while the corresponding matrix multiplication order is reversed; the word interpretation accounts for this by recursively lifting the tail and then composing the current box.

12 Box Relations

The public index of box relations is `BoxRelations.agda`. It is parameterized by $p - 2$ and a proof that $2 + p - 2$ is prime. It imports the symplectic presentation, the box interpretation, and the specific proof modules under `N/BR`. The file is organized into one-, two-, and three-qubit modules.

The one-qubit statements are:

$$\begin{aligned} [x]^A \cdot H &\approx \text{dir} \cdot [x']^A, \\ [x]^A \cdot S &\approx \text{dir} \cdot [x']^A, \\ [b]^E \cdot S &\approx [b - 1]^E. \end{aligned}$$

The actual direction word and updated box parameter are computed by functions in `N/BR/One/A.agda`; the `E` rule is proved in `N/BR/One/E.agda`.

The two-qubit statements are:

$$\begin{aligned} [l]^L \cdot CZ &\approx \text{dir} \cdot [l']^L, \\ [b]^B \cdot [g] &\approx \text{dir} \cdot [b']^B, \\ [d]^D \cdot [g] &\approx S^e \cdot \text{dir}^\uparrow \cdot [d']^D. \end{aligned}$$

Here the `B` rule ranges over the gates $H^\uparrow$, $S^\uparrow$, and S , excluding H and CZ by explicit inequality hypotheses. The `D` rule ranges over H , $S^\uparrow$, S , and CZ , excluding $H^\uparrow$.

The three-qubit statements are:

$$\begin{aligned} [vb]^{\bar{B}} \cdot CZ^\uparrow &\approx \text{dir}^\downarrow \cdot [vb']^{\bar{B}}, \\ [b]^{\text{B}\uparrow} \cdot CZ &\approx \text{dir} \cdot [b]^{\text{B}\uparrow}, \\ [vd]^{\bar{D}} \cdot CZ &\approx \text{dir}^\uparrow \cdot [vd']^{\bar{D}}. \end{aligned}$$

These are delegated to `N/BR/Three/BB-CZ.agda`, `N/BR/Three/B-CZ.agda`, and `N/BR/Three/DD-CZ.agda`, respectively.

13 Example: The E Rule

The README includes one compact derivation, and it is useful because it shows the style of the whole project. The `E` interpretation is $[b]^E = S^{-b}$. The box relation says:

$$[b]^E \cdot S \approx [b - 1]^E.$$

The proof in `N/BR/One/E.agda`, also shown in `doc/Readme.lagda.tex`, proceeds by unfolding the interpretation and using a lemma that combines powers of S :

$$S^{-b} \cdot S \approx S^{-b+1}.$$

Then finite-field arithmetic rewrites $-b + 1$ into $-(b - 1)$, which is exactly the exponent used by $[b - 1]^E$.

This example illustrates the division of labor in the repository. A box theorem is not a custom decision procedure. It is a typed calculation in the presentation closure, with proof steps supplied by previously proved lemmas about powers, modular arithmetic, and word associativity.

14 Generic Presentation Machinery

Although the top-level goal is about Clifford circuits, a significant part of the repository is generic. The modules under `Presentation/Construct` define ways to build larger presentations from smaller ones, including direct products, semidirect products, and amalgamations. The modules `Presentation/Coset` and `Presentation/Reidemeister-Schreier.agda` formalize coset and rewriting infrastructure. The older `Presentation/Groups` directory contains example and exploratory group presentations such as cyclic groups, symmetric groups, and Clifford one- and two-qubit fragments.

The Clifford development uses this infrastructure in two ways. First, the basic closure and setoid machinery are used directly throughout the proof scripts. Second, the normal-form and isomorphism modules reuse the general idea that a presentation can be related to another presentation by a well-defined star extension of a generator map. For a map $f : X \rightarrow \text{Word}Y$, the extension $f^* : \text{Word}X \rightarrow \text{Word}Y$ is implemented by mapping generators to words and then concatenating. Well-definedness means that each axiom of the source presentation maps to a derivable equality in the target presentation.

15 Artifact Boundary

The repository contains unfinished code, visible as Agda holes `{!!}` or `?` in several older or exploratory files. This is important for an artifact paper: the proof claims should be scoped to the modules that are intended to constitute the development. The README itself tells the reader to start from `doc/Readme.pdf` and `BoxRelations.agda`; the latter is the best public index for the box-relation statements.

This distinction is not merely administrative. Some files such as `N/Completeness.agda`, `N/Symplectic-Alternative.agda`, and older group-presentation experiments contain partially developed normal-form or alternative-presentation work. Those modules are valuable context, but they should not be cited as completed theorems unless they are closed and type checked in the release artifact. By contrast, modules marked `-safe` and free of holes can be treated as checked under Agda's kernel, subject to the standard library and the explicit postulates or options they import. For a POPL artifact, the natural packaging step is to provide a small driver module that imports exactly the intended theorem surface: `BoxRelations.agda`, the `simplified-presentation` isomorphism modules, and the finite-field modules used by them. Such a driver would make the checked boundary mechanically visible.

16 Trusted Computing Base

The trusted base of the development is conventional for an Agda formalization:

- the Agda type checker;

- the Agda standard library, tested in the README with Agda 2.8 and standard library 2.3, and also with Agda 2.7 and standard library 2.2;
- the definitions of the Clifford generators, relation families, and box interpretations;
- the mathematical adequacy claim connecting the formal generators and relations to the intended circuit diagrams.

The intended derivations themselves do not rely on an external computer algebra system. Modular arithmetic is proved inside the repository using `Fin`, natural-number modular arithmetic, and standard-library algebraic structures. Associativity normalization is also proved inside the presentation layer, by flattening words to lists and rebuilding them.

17 How to Read the Code

A reader who wants to audit the development can follow this path.

1. Read `Word/Base.agda` and `Presentation/Base.agda`. These define the syntax and equality relation used everywhere else.
2. Read `Zp/ModularArithmetic.agda` enough to understand $\mathbb{Z}_p$, nonzero elements, inverse notation, and ring lemmas.
3. Read the first half of `N/Symplectic.agda`: generators, derived words, $M(x)$, and the comprehensive relation family.
4. Read `N/Boxes.agda` and the beginning of `N/LM-Sym.agda`: these explain what boxes are and how they are interpreted as circuit words.
5. Read `BoxRelations.agda`. It is short and gives the theorem statements for the box relations.
6. Follow one proof pointer at a time into `N/BR`. The one-qubit `E` proof is the easiest starting point; the two- and three-qubit proofs rely on more associativity and commutation lemmas.
7. Read `N/Symplectic-Simplified.agda`, `N/Iso-Sym-Deriv`, `BoxRelations.agda`, and `N/Action-Sym.agda` for the connection between the paper-facing axiom set, derived generators, and semantic Pauli actions.

18 Related Work

The mathematical context is the theory of stabilizer circuits and the Clifford group in odd prime dimension. The symplectic description of qudit Cliffords used above goes back to the stabilizer formalism [3, 4] and its higher-dimensional extensions [5, 7, 8, 10]; the efficient classical simulation of stabilizer circuits [6] and discrete phase-space methods for odd dimensions [11] rely on the same linear-symplectic action. The splitting of the Clifford-to-symplectic sequence over a finite field is the Weil representation [12]. Magic-state and resource-theoretic results single out odd prime dimensions

as especially well behaved [13–15], which is the regime the present development targets.

Finite presentations of the Clifford group are the direct ancestor of the relation sets in `N/Symplectic.agda`. Selinger gave generators and relations for the n -qubit Clifford operators [16]; the constructors `selinger-c10-c15` are named after that style of two- and three-qubit relation. Farinholt gave an ideal characterization and presentation of the qudit Clifford operators [9]. Diagrammatic calculi offer an alternative complete equational theory: the ZX-calculus is complete for qubit stabilizer quantum mechanics [17, 18] and for the qutrit stabilizer fragment [19].

The proof-engineering context is mechanized reasoning about quantum programs and about algebraic presentations. Formal verification of quantum circuits has been carried out in Coq with `QWIRE` [20] and the `VOQC` verified optimizer [21], and via path-sum and related semantic methods [22]. The present work instead uses Agda [23, 24] and its standard library [2], in the setoid-based style where raw syntax is kept separate from the congruence that quotients it. The generic presentation layer—congruence closure, coset normal forms, and Reidemeister–Schreier rewriting—follows classical combinatorial and computational group theory [25–29]. The finite-field layer is standard [30, 31].

19 Conclusion

QUPIT is a mechanized proof development for Clifford-circuit box relations over odd prime dimensions. Its central move is simple and effective: define circuit syntax as binary words, define equality as the congruence closure of a presentation, and prove each diagrammatic rewrite as an Agda term in that equality. Around this core, the repository builds finite-field arithmetic, Clifford generators, simplified and derived relation sets, Pauli actions, box interpretations, and a collection of concrete box-relation proof scripts.

The current code base also records its own research history. Some older modules are unfinished, and a POPL artifact should make the intended checked surface explicit. The file `BoxRelations.agda` already provides the right shape for that surface: it states the one-, two-, and three-qubit box relations and points to the modules where their derivations live. This paper expands the existing README into a structured explanation of how those pieces fit together and what must be checked to support the artifact claims.

References

- [1] The Agda Team. Agda documentation. <https://agda.readthedocs.io/>. Accessed 2026-01-15 in the repository documentation.
- [2] The Agda Standard Library Team. Agda standard library documentation, version 2.3. <https://agda.github.io/agda-stdlib/v2.3/>. Accessed 2026-01-15 in the repository documentation.
- [3] D. Gottesman. Stabilizer Codes and Quantum Error Correction. PhD thesis, California Institute of Technology, 1997. arXiv:quant-ph/9705052.

- [4] D. Gottesman. The Heisenberg representation of quantum computers. In *Group22: Proc. XXII Int. Colloquium on Group Theoretical Methods in Physics*, pp. 32–43, 1999. arXiv:quant-ph/9807006.
- [5] D. Gottesman. Fault-tolerant quantum computation with higher-dimensional systems. *Chaos, Solitons & Fractals*, 10(10):1749–1758, 1999. arXiv:quant-ph/9802007.
- [6] S. Aaronson and D. Gottesman. Improved simulation of stabilizer circuits. *Physical Review A*, 70:052328, 2004. arXiv:quant-ph/0406196.
- [7] E. Hostens, J. Dehaene, and B. De Moor. Stabilizer states and Clifford operations for systems of arbitrary dimensions and modular arithmetic. *Physical Review A*, 71:042315, 2005. arXiv:quant-ph/0408190.
- [8] N. de Beaudrap. A linearized stabilizer formalism for systems of finite dimension. *Quantum Information & Computation*, 13(1–2):73–115, 2013. arXiv:1102.3354.
- [9] J. M. Farinholt. An ideal characterization of the Clifford operators. *Journal of Physics A: Mathematical and Theoretical*, 47(30):305303, 2014. arXiv:1307.5087.
- [10] D. M. Appleby. Symmetric informationally complete-positive operator valued measures and the extended Clifford group. *Journal of Mathematical Physics*, 46:052107, 2005. arXiv:quant-ph/0412001.
- [11] D. Gross. Hudson’s theorem for finite-dimensional quantum systems. *Journal of Mathematical Physics*, 47:122107, 2006. arXiv:quant-ph/0602001.
- [12] M. Neuhauser. An explicit construction of the metaplectic representation over a finite field. *Journal of Lie Theory*, 12(1):15–30, 2002.
- [13] V. Veitch, S. A. H. Mousavian, D. Gottesman, and J. Emerson. The resource theory of stabilizer quantum computation. *New Journal of Physics*, 16:013009, 2014. arXiv:1307.7171.
- [14] E. T. Campbell, H. Anwar, and D. E. Browne. Magic-state distillation in all prime dimensions using quantum Reed–Muller codes. *Physical Review X*, 2:041021, 2012. arXiv:1205.3104.
- [15] M. Howard, J. Wallman, V. Veitch, and J. Emerson. Contextuality supplies the ‘magic’ for quantum computation. *Nature*, 510:351–355, 2014. arXiv:1401.4174.
- [16] P. Selinger. Generators and relations for n -qubit Clifford operators. *Logical Methods in Computer Science*, 11(2):10, 2015. arXiv:1310.6813.
- [17] B. Coecke and R. Duncan. Interacting quantum observables: categorical algebra and diagrammatics. *New Journal of Physics*, 13:043016, 2011. arXiv:0906.4725.
- [18] M. Backens. The ZX-calculus is complete for stabilizer quantum mechanics. *New Journal of Physics*, 16:093021, 2014. arXiv:1307.7025.
- [19] Q. Wang. Qutrit ZX-calculus is complete for stabilizer quantum mechanics. In *Proc. QPL 2018, EPTCS 287:313–344*, 2019. arXiv:1803.00696.
- [20] J. Paykin, R. Rand, and S. Zdancewic. qWIRE: a core language for quantum circuits. In *Proc. 44th ACM SIGPLAN Symp. on Principles of Programming Languages (POPL)*, pp. 846–858. ACM, 2017.
- [21] K. Hietala, R. Rand, S.-H. Hung, X. Wu, and M. Hicks. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages*, 5(POPL):37, 2021. arXiv:1912.02250.
- [22] M. Amy. Towards large-scale functional verification of universal quantum circuits. In *Proc. QPL 2018, EPTCS 287:1–21*, 2019. arXiv:1805.06908.
- [23] U. Norell. Towards a practical programming language based on dependent type theory. PhD thesis, Chalmers University of Technology, 2007.
- [24] A. Bove, P. Dybjer, and U. Norell. A brief overview of Agda—a functional language with dependent types. In *Theorem Proving in Higher Order Logics (TPHOLs)*, LNCS 5674, pp. 73–78. Springer, 2009.
- [25] W. Magnus, A. Karrass, and D. Solitar. *Combinatorial Group Theory*. Dover, 2nd revised edition, 1976.
- [26] R. C. Lyndon and P. E. Schupp. *Combinatorial Group Theory*. Springer, 1977.
- [27] D. L. Johnson. *Presentations of Groups*. London Mathematical Society Student Texts 15. Cambridge University Press, 2nd edition, 1997.
- [28] C. C. Sims. *Computation with Finitely Presented Groups*. Encyclopedia of Mathematics and its Applications 48. Cambridge University Press, 1994.
- [29] D. F. Holt, B. Eick, and E. A. O’Brien. *Handbook of Computational Group Theory*. Chapman & Hall/CRC, 2005.
- [30] R. Lidl and H. Niederreiter. *Finite Fields*. Encyclopedia of Mathematics and its Applications 20. Cambridge University Press, 2nd edition, 1997.
- [31] K. Ireland and M. Rosen. *A Classical Introduction to Modern Number Theory*. Graduate Texts in Mathematics 84. Springer, 2nd edition, 1990.